



Laboratorio 5 Implementación y Visualización de Algoritmos de Búsqueda

ICC708-1: PROGRAMACION AVANZADA

Nombre: Bárbara Constanza Sanhueza Bombín.

Carrera: Ingeniería informática.

Fecha: 24-06-2026.

Docente: Profesor Ricardo Gacitúa B.

Índice

Introducción	3
Diseño e implementación	3
Implementación en grafos	4
Implementación en laberintos	4
Visualización	4
Pruebas y comparación de resultados	5
Escenario 1: Grafo	5
Escenario 2: Laberinto	5
Conclusión	6
Pruebas sobre grafos	7
BFS - Grafo ($A \rightarrow F$)	7
DFS - Grafo ($A \rightarrow F$)	7
UCS - Grafo ($A \rightarrow F$)	8
A* - Grafo ($A \rightarrow F$)	8
Pruebas sobre laberintos	9
BFS - Laberinto ($0,0 \rightarrow 4,4$)	9
DFS - Laberinto ($0,0 \rightarrow 4,4$)	10
UCS - Laberinto ($0,0 \rightarrow 4,4$)	11
A* - Laberinto ($0,0 \rightarrow 4,4$)	12

Introducción

En este laboratorio se implementaron y compararon los algoritmos de búsqueda BFS (Breadth First Search), DFS (Depth First Search), UCS (Uniform Cost Search) y A* sobre dos escenarios: grafos y laberintos.

El escenario de grafos se representa mediante una estructura de nodos conectados por aristas con costos asociados, permitiendo realizar búsquedas entre un nodo inicial y un nodo objetivo.

El escenario de laberintos se representa mediante una matriz bidimensional, donde cada posición corresponde a una celda del mapa. En esta representación, el valor 0 indica una celda libre y el valor 1 indica una barrera.

El sistema permite seleccionar el escenario, el algoritmo de búsqueda, establecer una posición inicial y una posición objetivo, mostrando posteriormente el orden de exploración, el camino encontrado y el costo de la solución.

Diseño e implementación

El proyecto fue desarrollado en Java con una organización por paquetes para separar las responsabilidades del sistema. La clase principal Main permite seleccionar el escenario de prueba (grafo o laberinto), el algoritmo de búsqueda, los puntos inicial y objetivo, y luego muestra los resultados obtenidos.

El paquete algorithms contiene las implementaciones de los algoritmos BFS, DFS, UCS y A* para ambos escenarios. Las clases GraphBFS, GraphDFS, GraphUCS y GraphAStar trabajan sobre estructuras de grafos, mientras que MazeBFS, MazeDFS, MazeUCS y MazeAStar realizan las búsquedas sobre matrices de laberintos.

El paquete models contiene las estructuras utilizadas durante la ejecución, como Graph, Node y Edge para representar grafos, además de Position para manejar coordenadas del laberinto. También incluye las clases SearchResult y MazeSearchResult, encargadas de almacenar los nodos visitados, el camino encontrado y el costo obtenido por cada algoritmo.

El paquete environments contiene los escenarios utilizados. GraphEnvironment permite definir el grafo de prueba mediante nodos y conexiones, mientras que Maze representa el laberinto mediante una matriz donde el valor 0 corresponde a una celda libre y 1 a una barrera.

Finalmente, el paquete utils incluye la clase MazeVisualizer, utilizada para mostrar de forma textual el recorrido del laberinto, destacando el inicio, objetivo, obstáculos y camino encontrado.

Implementación en grafos

El escenario de grafos fue implementado utilizando una estructura formada por nodos y aristas con costos asociados. Cada nodo se representa mediante la clase Node, mientras que las conexiones entre nodos se modelan mediante la clase Edge. La clase Graph permite administrar los nodos y obtener sus vecinos durante la búsqueda.

Para este escenario se implementaron los algoritmos BFS, DFS, UCS y A*. BFS utiliza una cola para explorar los nodos por niveles, mientras que DFS realiza una exploración en profundidad utilizando el recorrido por ramas. UCS utiliza una cola de prioridad para seleccionar siempre el nodo con menor costo acumulado. Finalmente, A* combina el costo recorrido con una función heurística mediante la fórmula $f(n)=g(n)+h(n)$ para priorizar los nodos más prometedores.

Los resultados de cada búsqueda se almacenan mediante SearchResult, que guarda el orden de nodos visitados, el camino encontrado y el costo final.

Implementación en laberintos

El escenario de laberintos fue implementado mediante una matriz bidimensional representada en la clase Maze. Cada posición de la matriz corresponde a una celda, donde el valor 0 indica una celda transitable y el valor 1 representa una barrera. Las posiciones inicial y objetivo son definidas mediante coordenadas utilizando la clase Position.

Los algoritmos BFS, DFS, UCS y A* fueron adaptados para trabajar con movimientos dentro de la matriz, validando que las posiciones sean recorribles y evitando atravesar obstáculos. En A* se utiliza la distancia Manhattan como función heurística para estimar la distancia restante hasta el objetivo.

Los resultados obtenidos se almacenan mediante MazeSearchResult y se muestran utilizando MazeVisualizer, permitiendo observar las celdas visitadas y el camino encontrado

Visualización

La visualización del recorrido se realiza mediante una representación textual del laberinto:

- S indica la posición inicial.
- G indica el objetivo.
- X representa barreras.
- * representa el camino encontrado.
- 0 representa espacios libres.

Esto permite observar el comportamiento de cada algoritmo y comparar la cantidad de posiciones exploradas.

Pruebas y comparación de resultados

Escenario 1: Grafo

Se utilizó un grafo compuesto por los nodos:

- A, B, C, D, E, F

con conexiones entre nodos. La búsqueda se realizó desde:

Nodo inicial: A y Nodo objetivo: F

Algoritmo	Camino encontrado	Costo	Observación
BFS	$A \rightarrow B \rightarrow D \rightarrow F$	8	Exploró por niveles hasta encontrar el objetivo
DFS	$A \rightarrow B \rightarrow D \rightarrow F$	3	Encontró una solución siguiendo profundidad
UCS	$A \rightarrow B \rightarrow D \rightarrow F$	7	Seleccionó el camino con menor costo acumulado
A*	$A \rightarrow B \rightarrow D \rightarrow F$	7	Encontró el camino óptimo utilizando la heurística

Escenario 2: Laberinto

Se utilizó una matriz de tamaño 5x5 donde:

- 0 representa una celda libre.
- 1 representa una barrera.

Matriz utilizada:

0	0	0	1	0
1	1	0	1	0
0	0	0	0	0
0	1	1	1	0
0	0	0	0	0

Puntos definidos:

Inicio: (0,0) y Objetivo: (4,4)

Algoritmo	Camino encontrado	Costo	Nodos visitados
BFS	$(0,0) \rightarrow (4,4)$	8	15
DFS	$(0,0) \rightarrow (4,4)$	12	13
UCS	$(0,0) \rightarrow (4,4)$	8	13
A*	$(0,0) \rightarrow (4,4)$	8	9

Se observa que BFS encuentra un camino mínimo en cantidad de pasos, pero explora más posiciones. DFS puede encontrar soluciones menos eficientes al priorizar la profundidad. UCS garantiza el menor costo acumulado y A* logra una solución óptima reduciendo la cantidad de nodos explorados mediante la heurística Manhattan.

Conclusión

La implementación realizada permitió comparar el comportamiento de los algoritmos BFS, DFS, UCS y A* en escenarios de grafos y laberintos. Los resultados demostraron que cada algoritmo aplica una estrategia diferente para encontrar soluciones.

En las pruebas realizadas sobre el laberinto, BFS encontró un camino óptimo con un costo de 8, pero necesitó explorar una mayor cantidad de posiciones antes de llegar al objetivo. DFS encontró una solución con mayor costo debido a que prioriza la exploración en profundidad. UCS obtuvo un camino óptimo considerando el costo acumulado, mientras que A* logró encontrar una solución de igual costo reduciendo la cantidad de posiciones visitadas gracias al uso de la heurística Manhattan.

La comparación permitió observar que el algoritmo más eficiente depende del escenario y del objetivo de la búsqueda. BFS es útil cuando se busca una solución por niveles, UCS cuando existen costos variables y A* cuando se busca optimizar la exploración utilizando información estimada. El proyecto cumple con la implementación de los algoritmos solicitados, la selección de escenarios, definición de puntos inicial y objetivo, obtención de resultados y visualización del recorrido.

Evidencias de ejecución

Pruebas sobre grafos

BFS - Grafo (A → F)

Ejecución de BFS sobre el grafo mostrando nodos visitados, camino encontrado y costo.

SEARCH ALGORITHMS

1. Graph
2. Maze

1

Available nodes:

A B C D E F

Start node: A

Goal node: F

Algorithm:

1. BFS
2. DFS
3. UCS
4. A*

1

Visited nodes:

[A, B, C, D, E, F]

Path:

[A, B, D, F]

Cost:

3

DFS - Grafo (A → F)

Ejecución de DFS mostrando el recorrido en profundidad, camino obtenido y costo.

SEARCH ALGORITHMS

1. Graph
2. Maze

1

Available nodes:

A B C D E F

Start node: A

Goal node: F

Algorithm:

1. BFS
2. DFS
3. UCS
4. A*

2

Visited nodes:

[A, B, D, F]

Visited nodes:

[A, B, D, F]

Path:

[A, B, D, F]

Cost:

3

UCS - Grafo ($A \rightarrow F$)

Ejecución de UCS considerando costos acumulados para encontrar el camino.

SEARCH ALGORITHMS

1. Graph
2. Maze

1

Available nodes:

A B C D E F

Start node: A

Goal node: F

Algorithm:

1. BFS
 2. DFS
 3. UCS
 4. A*
- 3

Visited nodes:

[A, B, C, D, E, E, F]

Path:

[A, C, E, F]

Cost:

7

A* - Grafo ($A \rightarrow F$)

Ejecución de A* utilizando la función heurística para encontrar la solución.

SEARCH ALGORITHMS

1. Graph
2. Maze

1

Available nodes:

A B C D E F

Start node: A

Goal node: F

Algorithm:

1. BFS
 2. DFS
 3. UCS
 4. A*
- 4

Visited nodes:

[A, B, C, D, E, F]

Path:

[A, C, E, F]

Cost:

7

Pruebas sobre laberintos

BFS - Laberinto (0,0 → 4,4)

BFS aplicado al laberinto mostrando posiciones visitadas, camino y visualización.

```
1 // Program to find shortest path using BFS
```

```
2 SEARCH ALGORITHMS
```

```
3 1. Graph
```

```
4 2. Maze
```

```
5 2
```

```
6 Maze dimensions:
```

```
7 Rows: 5
```

```
8 Columns: 5
```

```
9 Use positions from 0 to rows-1 / columns-1
```

```
10 0 = free, 1 = wall
```

```
11 Start row: 0
```

```
12 Start column: 0
```

```
13 Goal row: 4
```

```
14 Goal column: 4
```

```
15 Algorithm:
```

```
16 1. BFS
```

```
17 2. DFS
```

```
18 3. UCS
```

```
19 4. A*
```

```
20 1
```

```
21 Visited positions:
```

```
22 [(0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,3), (2,0), (2,4), (3,0), (1,4), (3,4), (4,0), (0,4), (4,4)]
```

```
23 Path:
```

```
24 [(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
```

```
25 Cost:
```

```
26 8
```

```
27 Maze visualization:
```

```
28 S * * X 0
```

```
29 X X * X 0
```

```
30 0 0 * * *
```

```
31 0 X X X *
```

```
32 0 0 0 0 G
```

DFS - Laberinto (0,0 → 4,4)

DFS aplicado al laberinto mostrando exploración en profundidad y resultado.

```
1 // Program to find shortest path using DFS
```

```
2 SEARCH ALGORITHMS
```

```
3 1. Graph
```

```
4 2. Maze
```

```
5 2
```

```
6 Maze dimensions:
```

```
7 Rows: 5
```

```
8 Columns: 5
```

```
9 Use positions from 0 to rows-1 / columns-1
```

```
10 0 = free, 1 = wall
```

```
11 Start row: 0
```

```
12 Start column: 0
```

```
13 Goal row: 4
```

```
14 Goal column: 4
```

```
15 Algorithm:
```

```
16 1. BFS
```

```
17 2. DFS
```

```
18 3. UCS
```

```
19 4. A*
```

```
20 2
```

```
21 Visited positions:
```

```
22 [(0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,0), (3,0), (4,0), (4,1), (4,2), (4,3), (4,4)]
```

```
23 Path:
```

```
24 [(0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,0), (3,0), (4,0), (4,1), (4,2), (4,3), (4,4)]
```

```
25 Cost:
```

```
26 12
```

```
27 Maze visualization:
```

```
28 S * * X 0
```

```
29 X X * X 0
```

```
30 * * * 0 0
```

```
31 * X X X 0
```

```
32 * * * * G
```

UCS - Laberinto (0,0 → 4,4)

UCS aplicado al laberinto buscando el camino de menor costo.

```
1 // Program UCS using Search (Depth-First)
```

```
SEARCH ALGORITHMS
```

```
1. Graph
```

```
2. Maze
```

```
2
```

```
Maze dimensions:
```

```
Rows: 5
```

```
Columns: 5
```

```
Use positions from 0 to rows-1 / columns-1
```

```
0 = free, 1 = wall
```

```
Start row: 0
```

```
Start column: 0
```

```
Goal row: 4
```

```
Goal column: 4
```

```
Algorithm:
```

```
1. BFS
```

```
2. DFS
```

```
3. UCS
```

```
4. A*
```

```
3
```

```
Visited positions:
```

```
[(0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,3), (2,0), (2,4), (3,0), (3,4), (1,4), (4,4)]
```

```
Path:
```

```
[(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
```

```
Cost:
```

```
8
```

```
Maze visualization:
```

```
S * * X 0
```

```
X X * X 0
```

```
0 0 * * *
```

```
0 X X X *
```

```
0 0 0 0 G
```

A* - Laberinto (0,0 → 4,4)

A* aplicado al laberinto usando heurística para optimizar la búsqueda.

```
#!/usr/bin/perl -w; use strict; use warnings;
```

```
SEARCH ALGORITHMS
```

```
1. Graph
```

```
2. Maze
```

```
2
```

```
Maze dimensions:
```

```
Rows: 5
```

```
Columns: 5
```

```
Use positions from 0 to rows-1 / columns-1
```

```
0 = free, 1 = wall
```

```
Start row: 0
```

```
Start column: 0
```

```
Goal row: 4
```

```
Goal column: 4
```

```
Algorithm:
```

```
1. BFS
```

```
2. DFS
```

```
3. UCS
```

```
4. A*
```

```
4
```

```
Visited positions:
```

```
[(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
```

```
Path:
```

```
[(0,0), (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), (4,4)]
```

```
Cost:
```

```
8
```

```
Maze visualization:
```

```
S * * X 0
```

```
X X * X 0
```

```
0 0 * * *
```

```
0 X X X * |
```

```
0 0 0 0 G
```